

Payments API Deprecation Playbook



A six step operating model for retiring legacy payment APIs

This playbook lays out how I would deprecate legacy payment APIs, covering the key decisions, trade-offs and execution needed to move merchants to the right long-term platform. **The goal is to create a consistent, repeatable deprecation process that minimizes merchant disruption and enables a safe transition to the new platform.**

Why this playbook: I spent 12 years building payment infrastructure at PayPal, including working on large-scale merchant API migrations. That experience taught me that the technical solution is often the easier part. The harder part is making a change that works for the business, merchants and engineering teams at the same time—while protecting payment reliability and maintaining trust through the transition.

Why payment API migrations are uniquely hard

1. Money moves through it. That changes everything.

When a payment API breaks, a merchant can lose transactions, revenue, and customers in real time. Unlike a typical API migration, merchants are being asked to change a working, business-critical integration, where the cost of something going wrong is immediate but the benefit of migrating may not be. This is why merchants often defer migration until there is a clear reason, a credible deadline, and confidence that the transition will not disrupt their payment flow.

2. A working API doesn't always mean a working payment flow.

A payment integration spans APIs, webhooks, order systems, reconciliation jobs, and internal business logic. A migration can therefore look successful at the API layer while still breaking the end-to-end payment flow—for example, a change in

payment status semantics could lead to incorrect order fulfillment or reconciliation.

3. The provider controls the deadline; the merchant controls the release.

Even when the new API is ready, merchants may need engineering, QA, security, compliance, and production approvals before deploying it. Large or regulated merchants may also have limited release windows. A migration that takes days to build can take months to safely deploy.

Step 1: Assess Before You Deprecate

Before planning the migration, **assess whether deprecating the API is worth the disruption to merchants.**

Look at both sides:

- **Cost of keeping it:** infrastructure, maintenance, support, security/compliance risk, and limitations on future product development.
- **Cost of retiring it:** engineering and migration effort, merchant effort, support load, and the risk of payment failures or merchant churn.

The output is a clear **Proceed / Wait / Do not deprecate** decision, backed by three answers:

1. **Why does the API need to go?**
2. **Why would merchants want to move?** Identify what the new API does better from the merchant's perspective—better reliability, fewer payment failures, new capabilities, simpler integration, or something else they actually value.
3. **Can merchants migrate safely?** Confirm that the replacement supports key use cases and understand any capability gaps or behavioral differences.

Only once the decision is “Proceed”, move to next steps.



Worked example: retiring Payments v1 (illustrative, carried through each step)

800 merchants are still on our legacy Payments v1 API, running ₹9,000 crore of annual TPV. Keeping it alive costs roughly 1.5 engineers of

sustained maintenance, its share of infrastructure and on-call, and it blocks the network tokenization work we want to ship next year. Retiring it will take roughly two quarters of our engineering time. Merchants will need anywhere from three days to four weeks of work, depending on how they are built. Support tickets will go up while it runs.

Decision: Proceed. v2 also gives merchants a measurably better authorization success rate, which answers "why would merchants want to move?" without us having to manufacture a reason.

Step 2: Segment the Merchant Base

I would **segment the merchants before setting a sunset date**. Setting the date first and then discovering who is affected is a common reason deprecations slip. Once a date is public, moving it also makes future deadlines harder to take seriously.

The goal is not to create a complicated segmentation. It is to understand **where the business risk is, who can migrate on their own, and where we need to help**.

For each merchant, I would look at:

- **Business impact:** TPV and API traffic, especially where a small number of merchants account for a large share of volume.
- **Integration complexity:** APIs used, webhooks, response fields and downstream dependencies such as payments, refunds and reconciliation.
- **Ability to migrate:** Whether they have their own engineering team, use an agency or plugin, or depend on a platform/aggregator.
- **Contractual constraints:** Notice periods, SLAs and other commitments that may affect the timeline.
- **Who controls the integration:** A merchant may not be able to make the change themselves if the integration is owned by a platform or third party.

This should give us a few useful groups:

Merchant group	How I would handle them
Large volume, own engineers	Named owner on both sides and regular check-ins
Platforms / aggregators	Engage early and align on their roadmap
Plugin / agency-built	Make migration as close to a plugin upgrade as possible
Small, simple integrations	Self-serve through API warnings, docs and reminders
Complex, low-volume	Consider an adapter rather than expensive one-to-one support

Set the date: Only after understanding these groups would I set the sunset date. For the merchants representing most of the volume or risk, I would validate how long migration will actually take, including development, testing, approvals and release cycles. I would also account for contractual notice periods. The goal is not to pick the earliest possible date. It is to pick a date we can commit to and hold.



Worked example, continued: segments and date

Group	Merchants	Share of TPV
Top direct merchants	18	78%
Platform partners	3 platforms, ~250 merchants behind them	14%
Plugin-based	400	6%
Long tail, complex	132	2%

Now the date. Our biggest merchant is a lender. They can only release once a quarter, and their contract gives them 90 days notice.

Realistically they need nine months to be live on v2.

Add time for the merchants who will start late, and the earliest date we can actually hold is twelve months out.

Step 3: Choose the Migration Path

Start with one question: **Do merchants actually need to change code, or can we retire the old API behind a compatibility layer?**

There are three broad options:

Path	Use when	Main trade-off
Direct migration	The new API is materially better and merchant effort is manageable	Higher merchant effort
Compatibility layer	Merchant changes are expensive and we can safely translate the old API to the new one	Keeps some legacy complexity alive
Hybrid	Most merchants can migrate, but a small high-risk group needs a different path	More operational complexity

For each option, look at how much work it creates for merchants, the risk to payments, and what it will cost us to build and maintain. Work with Engineering to understand the effort, timeline and technical challenges before making the trade-off.

A **compatibility layer** may increase our engineering and operational cost, but could significantly reduce merchant effort and payment risk. A **direct migration** may be simpler and cleaner in the long term, but shifts more engineering, testing, and production risk to merchants. A **hybrid approach** can reduce risk for high-impact merchants while allowing others to migrate directly, but comes with additional operational complexity.

The goal is not to pick the cleanest technical architecture. It is to choose the approach that **gets us to the target state with the lowest overall migration risk and least unnecessary disruption to merchants.**



Worked example: picking the path

We use three paths, not one.

The biggest merchants and the platforms have their own engineers, and v2 is genuinely better for them. They change their code.

About 400 merchants use our plugin. A new plugin version means they just upgrade.

The last group have small integrations that are awkwardly built. Helping each one individually would cost more than their volume is worth. They sit behind a compatibility layer and change nothing.

Step 4: Run the Migration Program

Once the migration path and merchant segments are clear, the PM's job shifts from deciding what should happen to making sure the migration actually happens safely.

1. Prepare Before You Announce

Before announcing the sunset timeline, make sure we are internally ready with everything a merchant needs to move from the legacy integration to the new one. This includes:

- **Migration guide** — clear steps, code examples and old vs. new behavior
- **Sandbox & testing** — a working environment to test the new integration safely
- **Migration tooling** — SDKs, plugins, scripts or other tools that reduce merchant effort
- **Merchant impact report** — what each merchant uses today, what changes and what they need to do
- **Support & escalation** — clear owners, FAQs and a process for handling migration issues
- **Plan for existing payments** — Payments made through the legacy API may still need refunds, disputes, cancellations and reporting long after new payments have moved to the new integration.

2. Build a Communication Plan That Drives Action

Treat communication as a **migration mechanism, not an announcement**. The common mistake is sending one email and assuming the right person will see it. That creates awareness, but not necessarily action.

For each merchant, identify these roles:

- **Engineer**- needs to execute and incorporate the changes.
- **Technical Lead/ EM** — needs to understand exactly what changes and how much work is involved and plan work into a sprint or release.
- **Release Owner** — needs to account for testing, approvals and deployment windows.

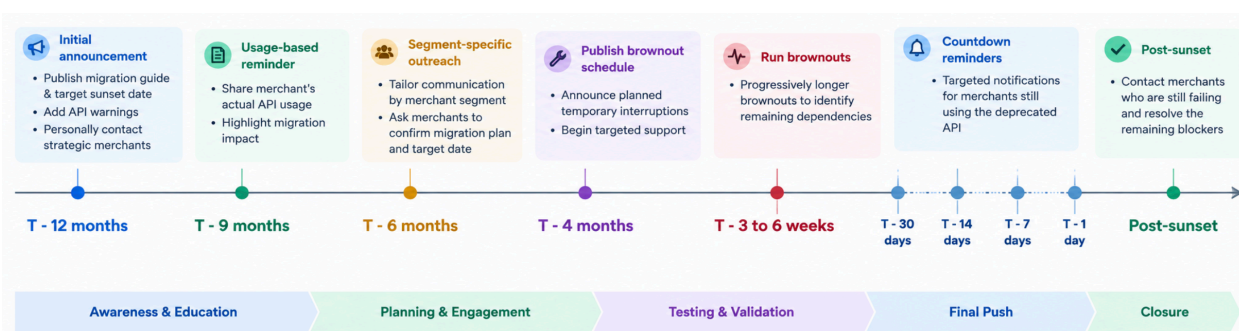
- **Account Owner** — can create internal urgency when migration is stuck.

Do not rely on a single contact or channel. Assume contact lists are incomplete and design for **redundant communication across technical, operational and commercial channels**.

The communication should progressively move merchants through: **Awareness** → **Planning** → **Commitment** → **Migration** → **Escalation** → **Enforcement**.

Intervention	Audience	Purpose
Deprecation announcement — email, docs, changelog, API warning	Technical, operational & business stakeholders	Create awareness
Migration guide, SDK/plugin, sandbox	Engineers, Technical Leads, Release Owners	Enable migration
Account outreach	Business / Account Owner	Establish ownership and timeline
Office hours / technical support	Engineers, Technical Leads	Remove blockers
Progress reviews / reminders	Technical + Account Owners	Drive completion
Escalation / final notice	Technical, business & operational owners	Create urgency
Enforcement notice	All stakeholders	Complete migration

My recommended cadence is illustrated below:



The exact cadence can change based on the migration window, but the principle should remain: **early communication creates awareness; repeated, targeted communication creates action.**

3. Drive Migration Through Progressive Enforcement

The announcement starts the migration; **enforcement is what drives it to completion**. Do not wait until the sunset date and switch the API off. Increase the level of intervention as the deadline approaches using the below model:

1. Warning: Make the deprecation visible wherever merchants interact with the API —API responses, emails, dashboards and documentation. Clearly communicate the sunset date and required action.

2. Friction: Make continued use of the legacy API increasingly inconvenient without immediately putting payments at risk. Examples include removing new feature access, limiting support, requiring explicit acknowledgement or requiring an approved exception to continue.

3. Brownouts: Introduce short, pre-announced periods where the legacy API is unavailable. Use these to identify merchants that have not migrated and validate that the new integration works at scale. For payment APIs, brownouts should be carefully controlled and risk-assessed.

4. Controlled Cutoff: Stop specific legacy functionality or traffic based on the agreed migration plan rather than switching everything off at once. Prioritise low-risk traffic first and maintain explicit exceptions where required.

5. Retirement: Once remaining traffic is understood and exceptions are resolved, permanently retire the legacy API.

Throughout the process, track **remaining TPV, traffic, merchant risk, migration blockers and payment health**. Use this data to decide when to move to the next enforcement stage and where additional intervention is required.

Enforcement should be a **cross-functional decision**, with Product owning the framework and Engineering, Support, Account and relevant Risk/Compliance teams providing operational sign-off.



Worked example: running it

We announce at T-12. The migration guide, the sandbox and the per-merchant reports are all live before we send anything.

By T-6, merchants making up 71% of our volume have told us when they will move. Two of the three platforms have given us a date. The third has not, so we stop emailing their engineers and their partnership lead picks it up instead.

Warnings start on day one. At T-4 we stop giving v1 users access to new features. The first brownout is at T-6 weeks.

Step 5: Measure Progress and Risk

Use **two connected dashboards**: one for leadership to understand overall progress and risk, and one for account, support and migration teams to manage merchant-level execution.

Together, they should answer three questions: **How much has moved? What is still at risk? Is the migration affecting payment health?**

1. Leadership Dashboard

Track **leading indicators, migration outcomes and payment guardrails**.

Leading indicators

- % affected TPV with committed migration dates
- % merchants that have started testing
- % merchants with production traffic
- Migration velocity vs. required pace

Migration outcomes

- % TPV migrated
- % merchants migrated
- Legacy vs. replacement traffic

Payment guardrails

- Payment success-rate delta
- API errors and latency

- Webhook failures
- Refund failures
- Reconciliation issues

Use **% TPV migrated as the primary progress metric**, but do not rely on it alone. Volume is a lagging indicator, so leading indicators such as **committed migration dates, and test traffic** should show whether the initiative is actually on track.

2. Merchant Dashboard

This is the working view for Account, Support, Engineering and Migration teams. It should answer: **What is happening with this merchant, what is blocking them, and what needs to happen next?**

Track:

- Merchant and account owner
- Deprecated and replacement API traffic
- % traffic migrated
- Migration stage
- Committed and actual migration dates
- Technical owner
- Blockers and risk
- Exception status
- Next action

The dashboard should be the **single source of truth** for merchant-level progress and should be reviewed regularly with Engineering, Support and Account teams.



Worked example, continued: what the dashboard shows at T-3

88% of merchants have migrated, but only 62% of TPV has. Two of the top 18 account for almost the entire gap. On merchant count the

programme looks nearly finished; on volume it is nowhere near ready for a cutoff. This is exactly why TPV is the primary metric. Payment guardrails are flat — success rate, webhook failures and refund failures all within baseline on v2 traffic.

Step 6: Plan for Exceptions, Rollback and Sunset

Plan for exceptions

Some merchants will not be ready by the sunset date. Allow extensions only for **genuine blockers** such as regulatory, contractual or critical technical dependencies.

Every exception should have:

- A clear reason
- A named owner
- A new migration date
- A mitigation plan

Track exceptions separately so you always know **how much TPV is still at risk and why**.

An exception should be temporary, owned and time-bound.

Plan for rollback

Before enforcement, agree with Engineering **what you will do if the migration causes unexpected payment impact**. The rollback plan should answer:

- **When do we roll back?** — Define the payment or operational signals that would trigger a rollback.
- **What do we roll back?** — Be clear whether you can roll back a merchant, cohort, API capability or all traffic.
- **Can we safely roll back?** — Make sure the legacy path can still support the traffic and that rollback will not create duplicate payments, lost transactions or

reconciliation issues.

- **Who decides?** — Agree upfront who can pause enforcement or trigger a rollback.
- **When do we resume?** — Define what needs to be fixed and validated before continuing.

Test the rollback before the final cutoff. A rollback plan is only useful if you know it will work when you need it.

Plan for final sunset

Execute the final cutoff like a high-risk production change. Before switching off the legacy API, confirm:

- Remaining traffic and dependencies are understood
- High-risk TPV has migrated
- Exceptions are closed or approved
- Rollback is tested and available
- Engineering, Support and on-call teams are ready
- Final communication has gone out

Then execute the cutoff in a controlled low-volume window and closely monitor the impact.



Worked example, continued: exceptions and cutoff

One of the top 18 merchant needs eight more weeks because of a compliance audit they cannot move. They get a written extension with a fixed date and a named owner. We do not move the sunset date for everyone else.

The brownout at T-6 weeks was worth doing. It found a merchant who had moved their payments to v2 months earlier but left refunds pointing at v1. Nothing in our tracking showed this, because their payment traffic on v1 had already dropped to zero. Without the brownout we would have found out after the cutoff, when a customer asked for their money back.

We switch off on a Sunday, when volume is lowest, and test the rollback the week before. A week later there is no traffic left on v1 — apart from the refund and dispute endpoints, which we kept open on purpose for older payments.

The next section looks at how leading payment platforms approach **API versioning and deprecation**, and what we can learn from their models.

Appendix

Comparative teardown: API versioning and deprecation

The approach above is designed around three things: **minimizing merchant disruption, making the migration predictable, and retiring the legacy API safely**. In this section, I look at how leading payment platforms handle API versioning and deprecation today, and what we can learn from their approach.

I'm comparing **Stripe, Razorpay, and Cashfree's API versioning and deprecation approaches** based on their public documentation and changelogs. I'm looking at what each platform does well, where their approaches differ, and whether those differences are reasonable trade-offs or areas that could be improved.

Dimension	stripe	Cashfree Payments	Razorpay
 Versioning model	Dated + named versions, e.g. <code>2026-08-26.dahlia</code>	Dated versions, e.g. <code>2026-01-01</code>	API-level versions, e.g. <code>/v1/</code>
 Merchant control	Per-request version or account default	Per-request version	No merchant-level version selection
 Release cadence	Monthly backward-compatible releases; 2 major releases/year	Annual dated versions	Ad hoc releases documented through changelog
 Breaking changes	Clearly defined and documented	Explicitly identified per version	Backward compatibility with deprecation warnings
 Webhook versioning	Independently pinned per endpoint	Follows API version	Not separately versioned
 Rollback / recovery	Self-serve rollback within 72 hours	Revert version header and redeploy	Not applicable
 Key strength	Strong merchant control and predictability	Clear versioning and breaking-change documentation	Simpler, API-level migration model

Stripe

Developers can manage the account's API version in **Workbench**, test newer versions per request using the Stripe-Version header, and upgrade the account only when ready.

Webhooks follow the account version by default, but can be pinned to a specific version during webhook endpoint creation in case merchants need more time to update its webhook consumer.

Stripe keeps releases predictable: **monthly releases are backward-compatible, while major releases twice a year may include breaking changes**, reducing the need for frequent breaking migrations.

For major upgrades, Stripe provides a **72-hour rollback window** after upgrading in Workbench, giving merchants a defined safety net to recover from production issues.

What I would change:

- Stripe is strong at supporting older API versions, but less explicit about when they must be retired. **The cost is permanent compatibility code and a long tail of old versions with degraded behavior.** I'd add a clear **supported** → **deprecated** → **unsupported** lifecycle with long, published timelines.
 - I'd provide a **merchant-level impact report** showing which APIs they use, which breaking changes affect them, and what they need to change—turning **“figure out which of these 20 changes matter”** into **“these 2 affect you, and here's what to fix.”**
-

Razorpay

Razorpay primarily versions APIs through the **API path**, with most APIs under /v1 and some newer ones under /v2. Unlike Stripe, there is no single merchant-wide API version; versioning is generally handled at the **API/product level**.

When Razorpay introduces a new API contract, it typically provides a **new endpoint/version and a migration path**. For example, for [Payment Links](#), Razorpay announced a new API version, gave a deprecation date for the older contract, and asked merchants to update their integrations.

I think this is reasonable for Razorpay because much of its base consists of **SMBs and agency-built integrations**. API-level versioning keeps migrations simpler: merchants only update the API that changed instead of testing their entire integration for every upgrade.

All Razorpay APIs are backwards-compatible. If an API or its parameters are deprecated, they add a warning note for the same for a specific period of time.

What I would change:

- I would give webhooks their own versioning mechanism, so that a change in webhook payloads doesn't become an involuntary breaking change for existing integrations.

- I'd also add machine-readable deprecation signals (in api response/response headers) and a merchant-level view showing which deprecated APIs they use, their traffic, and exactly what they need to change.
-

Cashfree

It follows a model similar to Stripe, with **dated API versions, an x-api-version header, and the previous version's documentation kept available**. It also clearly calls out breaking changes—for example, warning that the 2026-01-01 settlement schema change could cause errors or silent failures after upgrading.

What I would change:

- Publish a **clear support and sunset timeline** for every API version so merchants know how long they can stay on an older version.
- Ensure **SDK examples use currently supported versions**, so developers don't accidentally start with outdated versions.